PUBLIC

Revert StableSwap Hooks — MIESC audit triage

Target: revert-finance/stableswap-hooks @ cf0c30e5

Scope: Cantina competition — first-party src/ (12 files, ~1,348 nSLOC)

Scan: 0 Critical · 6 High · 17 Medium · 20 Low · 33 Info

Verdict: all 6 scanner Highs are false positives; the one judged High (#29, removeLiquidity reentrancy) was located by Slither but ranked Medium — static locates, it cannot adjudicate

MIESC — smart-contract security audit tooling · dogfooding case study. Triage layer over a default static scan. Public information from public repositories only; static analysis only; no exploitation; nothing submitted to any program.

Revert StableSwap Hooks — MIESC audit triage memo

Target: `revert-finance/stableswap-hooks` @ `cf0c30e5` · Cantina competition scope (first-party `src/`, 12 files / ~1,348 nSLOC) **What it is: StableSwap Hooks**, a Uniswap v4 hook that brings **Curve-style StableSwap** AMM behaviour to v4 pools — a constant-sum/constant-product blend with an amplification coefficient `A` (ramped over time), targeting stable assets (stablecoins, LSTs) for low-slippage swaps around the peg. It manages liquidity internally with Uniswap-v2-style ERC20 LP tokens (no NFT positions), supports per-asset rate oracles for non-1:1 pegs, and configurable LP/hook/protocol fees. A `StableSwapHooksFactory` deploys hooks via CREATE2 with off-chain salt mining and a creation-code-hash check; a `StableSwapZapIn` periphery enables single-token deposits by internally balancing across pool currencies. Swaps, liquidity and fees all route through the v4 `PoolManager.unlock` → `unlockCallback` lock. **Toolchain:** Foundry / **solc 0.8.30** (optimizer 200) + **Slither 0.11.3**, reusing forge build-info. **Scan (contest scope = `src/`): 76 owned findings — 0 Critical / 6 High / 17 Medium / 20 Low / 33 Info**. **Method note:** MIESC ran Slither with its **default detector set** over the whole first-party `src/`. 122 dependency results under `lib/` were excluded — including **3 `lib/` Highs** (1 `incorrect-exp` + 2 `incorrect-shift`), all Uniswap-v4-core / solmate assembly-math FPs; **none an `arbitrary-send-*`**.

> Triage layer. **Nothing here is a submission.** No PoC was built or submitted. The Cantina competition on this commit is **complete**, and the project publishes its **own findings log** (`CANTINA.md`, committed in-repo, with each finding, severity and fix), so this write-up carries no live-competition embargo; it reconciles MIESC's output against that public log. Any disclosure is the user's own action.

Headline

All six scanner Highs are false positives — but this is the sharpest "the scanner located the bug yet could not judge it" case in the whole series. The competition validated **12 findings: exactly one High (#29) and eleven Mediums** (per the project's committed `CANTINA.md`). The one High, #29, is a **native-ETH `removeLiquidity` reentrancy**, and it is present at the scanned commit — and Slither flagged **the exact function it lives in**, `Liquidity._handleRemoveLiquidityCallback`, as a reentrancy site. But it recorded it as **`reentrancy-no-eth` at *Medium***, not as the High it is, because the ETH leaves through `poolManager.take(currency, sender, amount)` — a plain external call with no `{value}` — so the `reentrancy-eth` detector never fires and the detector cannot see that `take` is an attacker-directed ETH exit. Meanwhile the scanner's *only* High-rated reentrancy sits on the **other** callback (`_handleAddLiquidityCallback`), which was **not** a judged finding at all. So the static layer put a flag on the one real High in scope, under-ranked it, over-ranked a false positive next to it, and was **completely blind to the other eleven findings** — every one a StableSwap-economics invariant (rounding asymmetry, a missing fee floor, invariant overflow, zap-in sizing, exact-output rounding) with no detector class. **Tenth target in a row with 0 exploitable scanner Highs** — and the clearest demonstration yet that a static scan can *locate* a vulnerability class and still not *adjudicate* severity or exploitability.

High findings — per-site verdict (6/6 FP)

1. `arbitrary-send-erc20` → FP · `Liquidity._handleAddLiquidityCallback` L172

`safeTransferFrom(sender, address(poolManager), actualAmounts[i])` pulls `sender`'s tokens. `sender` is **not attacker-chosen**: `addLiquidity` encodes `msg.sender` into the unlock payload (L85), and the callback runs only via `poolManager.unlock` → `unlockCallback`, which is `onlyPoolManager` (`StableSwapHooks` L40). An attacker cannot forge an `unlock` carrying someone else's `sender` — their own `unlock` would invoke *their own* contract's callback, never this hook's. So the pull always moves the real caller's own tokens into the PoolManager. FP.

2. `arbitrary-send-eth` → FP · `StableSwapZapIn._addLiquidityAndGetShares` L429

`_hooks.addLiquidity{value: nativeValue}(...)` sends ETH to `_hooks`, a caller-supplied pool. `nativeValue = balances[nativeIndex]` is the **periphery's own balance** (`address(this).balance`) — the transient funds this caller sent in for the zap — deposited into the pool the caller chose; temporary approvals are then revoked (L432-436) precisely so a user-supplied hook keeps no spending power. It moves the caller's own transient funds to the caller's own target, not other users' funds (the only edge is pre-stranded balance in the periphery, which the team tests for separately — STAB-7). FP.

3. `encode-packed-collision` → FP · `StableSwapHooksFactory.deploy` L112

`abi.encodePacked(_creationCode, abi.encode(poolManager, _currencies, …))`. A packed-encoding collision needs two or more *variable* dynamic fields whose boundary an attacker can shift. Here the first field `_creationCode` is **pinned** — `keccak256(_creationCode) == creationCodeHash` (L108) forces it to one exact blob — and the tail is a single `abi.encode(...)` (length-prefixed, injective). No boundary is movable, so no collision is reachable. FP.

4 & 5. `incorrect-exp` / `incorrect-shift` → FP · `StableSwapMath.cbrt` L217-238

Both fire inside `cbrt`, which is lifted **verbatim from Solady** (`FixedPointMathLib`) and is **formally verified** (the xuwinnie proof, cited in the NatSpec). The flagged `xor(...)` and `shl` / `shr` are intentional bit-manipulation in the log2-estimate + Newton-Raphson cube root, not a mistaken `**` or shift. Known-good library assembly. FP (same class as the 3 excluded `lib/` Highs in v4-core/solmate).

6. `reentrancy-eth` → FP · `Liquidity._handleAddLiquidityCallback` L132-187 *(the interesting one)*

Slither's flagged external call is `poolManager.settle{value: actualAmounts[i]}()` (L169), with `reserves[i] += actualAmounts[i]` written after it (L177). But that `settle` sends ETH to the **PoolManager** — trusted v4 core, which does not call back into the hook — so it cannot reenter. The only attacker-reachable ETH send is the refund `Address.sendValue(payable(sender), refundValue)` at **L183**, which runs *after* the loop has committed every `reserves[i]` and after `_mint(sender, newShares)` — a reentry there sees fully consistent state. The ERC20 pull (L172) could only call back on an ERC-777-style token, and these are stablecoin/LST pools. So at this commit there is no stale-state window on add-liquidity. **Not a judged finding** — the team nonetheless applied the CEI reorder here later (PR #131, "commit reserves before settlement") as hygiene after #29 taught the lesson. FP, correctly on a reentrancy-shaped pattern.

Where the static lens ends — the one judged High, and the eleven it never saw

The judged High (#29) — located, but ranked Medium

`CANTINA.md` records exactly one High: **#29, "Native ETH removeLiquidity reentrancy lets LP swap against withdrawn stale reserves."** It is present at the scanned commit, in `_handleRemoveLiquidityCallback`:

```
for (uint256 i = 0; i < currenciesLength; ++i) {
    ...
    poolManager.burn(address(this), currency.toId(), amounts[i]);
    poolManager.take(currency, sender, amounts[i]);   // sends out (ETH to `sender`)
    reserves[i] -= amounts[i];                          // books updated after the send
}
_burn(sender, shares);                                  // shares burned after the loop
```

When a currency is native ETH, `poolManager.take(currency, sender, amount)` performs a raw ETH transfer to `sender` from inside the PoolManager — handing control to the LP while the pool's books are still un-updated: `reserves` has not been decremented (for this currency or the later ones) and the LP's shares have not been burned (`_burn` runs only after the whole loop). The reentrant LP can then act against those **stale, still-large reserves and over-large share supply**, draining the pool at an unfair price — the project's own words: the pool "sent the ETH out **before updating its own books**." The fix was the CEI reorder: update all internal accounting and burn the shares first, send funds out last.

Slither **did** flag this exact function — but as `reentrancy-no-eth` **(Medium)** plus a `reentrancy-events` (Low). The under-classification is mechanical: the ETH does not leave via a `.call{value}` / `.transfer` the detector recognises, it leaves via `poolManager.take(currency, sender, amount)` — an ordinary external call. `reentrancy-eth` therefore never fires, and the finding is recorded as a generic "state write after external call" at Medium. The detector put a marker on the one real High in the scope; it could not see that `take` is a native-ETH exit to an attacker-controlled address, which is the entire reason the finding is High rather than Medium. That gap — *is this reentrancy a drain or a benign post-call write?* — is semantic, and it is exactly the adjudication the triage layer exists to perform.

The eleven Mediums — no detector class at all

Every other judged finding is **StableSwap economics**, invisible to any pattern detector:

#	Judged finding (Medium)	Why no detector sees it
546 / 522	Zero-fee pools: reserves extracted on round trips through StableSwap **rounding asymmetry** (546), or drained/bricked because the net-LP-fee floor that keeps reserves off zero is absent at 0% fee (522)	Economic invariants (rounding asymmetry, a fee floor); no code-shape to match
998 / 8	Exact-output swaps round the required input down to **zero** (free output) in reachable states / low-decimal pairs	Rounding-direction correctness in the swap math
102	Asymmetric exact-output math → systemic **fee evasion** / LP yield loss	Fee-math correctness
63 / 788	StableSwap invariant **overflow / non-convergence** bricks swaps in imbalanced pools	Numerical convergence of the Newton solver
53	Integer rounding in 3-token `geometricMean` → mispriced initial invariant	Rounding in initial-share math
512	`StableSwapZapIn` balancing **overshoots** target ratio in 3+ token pools (~17% LP loss)	Missing divisor in a sizing formula
1351 / 15	Zap-in quote understates shares / internal swaps lack slippage bounds → weak protection, sandwich	Quote-vs-execution consistency; slippage policy

None of these is an `arbitrary-send`, `reentrancy`, `uninitialized` or any other shape Slither models. They are the invariants of a Curve-style AMM — exactly the semantic surface a static scan cannot reach.

Reconciliation against the project's published findings log

`CANTINA.md` (committed in-repo) lists the judged outcome: **1 High (#29) + 11 Mediums**, each with a fix. Mapping MIESC's scan onto it:

- **#29 (High, removeLiquidity reentrancy):** Slither **located the function** (`_handleRemoveLiquidityCallback`) but ranked it **Medium** (`reentrancy-no-eth`) — the native-ETH `take` exit is invisible to `reentrancy-eth`. The real High was located but **mis-ranked**: a true positive at the location level, under-adjudicated on severity.
- **The 11 Mediums:** StableSwap-economics / rounding / fee / invariant findings — the static scan surfaced **none** of them.
- **MIESC's 6 Highs:** all false positives (above); the one reentrancy it rated High (`_handleAddLiquidityCallback`) is **not** among the judged findings.

This is the public leaderboard's shape too — multiple entrants scored a High (the judged #29) and many scored Mediums. MIESC's static layer did not produce a submittable finding; its signal on the real High was present but mis-ranked, and the economic majority was out of reach.

Bottom line

- **No submittable bug** from static scan + triage — **tenth** target in a row (Lombard, Doppler, Valantis STEX, YieldNest, Arcadia, v4lend, Malda, Royco Dawn, Reserve Governor, **Revert StableSwap Hooks**), all raw Highs being false positives.

- **All six scanner Highs are FPs:** two `arbitrary-send-*` on caller-own / caller-chosen funds behind the `onlyPoolManager` unlock callback, one `encode-packed-collision` on a hash-pinned creation code, two `incorrect-exp` / `-shift` on Solady's formally-verified `cbrt`, and one `reentrancy-eth` on add-liquidity whose only reentrant ETH send runs after state is committed (the team hardened it anyway, PR #131; not a judged finding).
- **The real residual risk is where the lens ends — and here the lens got *closest*:** the one judged **High (#29)** is a native-ETH `removeLiquidity` reentrancy that Slither flagged **in the exact function** but ranked **Medium**, because the ETH exits via `poolManager.take` rather than a recognised value-send. The remaining **eleven Mediums** are StableSwap-economics invariants (rounding asymmetry, fee-floor, invariant overflow, zap sizing, exact-output) with no detector class. Static **locates** a reentrancy; it cannot **adjudicate** whether it drains the pool, and it cannot see the economic findings at all.
- **Honest limits:** this is the **public contest commit** after 1,412 submitted findings, reconciled against the project's **own committed findings log**; verdicts are from reading the code; **no PoC was built or run.**

Appendix — Evidence trail

- Normalized scan: `results.json` (owned = first-party `src/` ; `lib/` dependencies excluded — the 3 excluded `lib/` Highs are Uniswap-v4-core / solmate `incorrect-exp` / `incorrect-shift` assembly-math FPs, no `arbitrary-send`).
- Reports: `triage-focused.pdf` (focused triage); the full premium PDF is regenerable from `results.json` and is not committed.
- Toolchain: HTTPS clone @ `cf0c30e5` + recursive submodules (`uniswap-hooks` → `v4-periphery` / `v4-core` / `permit2`) → `forge build --root <repo> --build-info --skip test/script` (solc 0.8.30) → `slither <repo> --foundry-ignore-compile --json` (Slither 0.11.3, default detectors). Build/scan were invoked from a safe working directory with the repo passed by absolute path (never `cd` -ing into the untrusted clone); Slither's `filename_relative` paths carry a `ss-audit/repo/` prefix, which the normalizer strips before scope-matching.
- Reconciliation source: `CANTINA.md` at `revert-finance/stableswap-hooks` default branch — the project's own public post-audit findings log (12 findings: 1 High + 11 Mediums, with fixes).
- Target pin: `revert-finance/stableswap-hooks` @ `cf0c30e576f144809df9819f4d3ad49e0b7fe2d7` (Cantina "StableSwap Hooks" competition, 30 Apr–7 May 2026).
- Only **public source from the public GitHub repository** and the project's **own public findings log** were accessed; **static analysis only**, no live systems / RPC / credentials, no exploitation, nothing submitted anywhere.